Reto 2 – Orquestación de Servicios y Persistencia de Datos

Contexto

Este reto continúa el desarrollo del **sistema de onboarding y offboarding de empleados**. En el reto anterior se construyó un servicio básico para la gestión de empleados y su despliegue en un contenedor Docker.

En este segundo reto se avanzará hacia una arquitectura más realista, incorporando:

- Orquestación de múltiples contenedores con **Docker Compose**
- Persistencia de datos mediante una **base de datos containerizada**
- Comunicación entre **servicios independientes**
- Gestión de **configuración mediante variables de entorno**

Objetivo

Evolucionar el sistema desarrollado en el reto anterior hacia una arquitectura de múltiples servicios orquestados, donde cada componente se ejecute de forma aislada en su propio contenedor, se comuniquen entre sí mediante la red interna de Docker, y los datos persistan más allá del ciclo de vida de los contenedores.

Requisitos generales

La solución desarrollada debe cumplir con los siguientes requisitos:

- Definir la infraestructura completa en un archivo `docker-compose.yml`.
- Implementar al menos **dos servicios de negocio** que se comuniquen entre sí.
- Incluir una **base de datos** como servicio independiente para cada microservicio.
- Garantizar la **persistencia de datos** mediante volúmenes Docker.

- Utilizar **variables de entorno** para la configuración de los servicios.
- Documentar los endpoints de ambos servicios utilizando **OpenAPI (Swagger)**.
- Permitir el despliegue completo mediante un único comando.

1. Docker Compose

Crear un archivo `docker-compose.yml` , en este archivo se debe definir toda la infraestructura:

Servicios a incluir

```yaml
services:
  empleados-service:
    # Servicio de gestión de empleados

  departamentos-service:
    # Servicio de gestión de departamentos

  database-empleados:
    # Base de datos (PostgreSQL, MySQL, MongoDB, etc.)

  database-departamentos:
    # Base de datos (PostgreSQL, MySQL, MongoDB, etc.)
```

Inicialmente empiece por registrar en su docker compose el servicio de empleados desarrollado en el reto anterior.

Requisitos del compose

Aspecto	Requisito
Redes	Definir una red interna para la comunicación entre servicios
Volúmenes	Configurar volumen para persistencia de la base de datos
Variables de entorno	Configurar credenciales y URLs de conexión
Dependencias	Usar `depends_on` para ordenar el inicio de servicios
Puertos	Exponer solo los puertos necesarios al host

Comando de despliegue

La solución debe poder desplegarse con:

```
docker-compose up --build
```

Y detenerse con:

```
docker-compose down
```

2. Servicio de Departamentos (nuevo servicio)

Implementar un nuevo microservicio encargado de la gestión de departamentos.

Endpoints requeridos

Método	Ruta	Descripción
`POST`	`/departamentos`	Registra un nuevo departamento
`GET`	`/departamentos/{id}`	Consulta un departamento por su identificador
`GET`	`/departamentos`	Lista todos los departamentos

Estructura del departamento

```
{
  "id": "string",
  "nombre": "string",
  "descripcion": "string"
}
```

Respuestas esperadas

- **Registro exitoso:** `201 Created` con el departamento creado.
- **Consulta exitosa:** `200 OK` con la información del departamento.
- **Departamento no existe:** `404 Not Found` con mensaje descriptivo.

Contenerización de la aplicación

- Debe crear un dockerfile para el servicio de departamentos.

Base de datos

- Debe crear una base de datos para el servicio de departamentos.
- Debe crear un volumen para la base de datos.
- Debe crear variables de entorno para la configuración de la base de datos.

Consideraciones

- El servicio de departamentos debe ser independiente del servicio de empleados.
- El servicio de departamentos debe tener su propia base de datos.
- El servicio de departamentos debe tener su propio puerto.
- El servicio de departamentos debe tener su propio volumen.
- El servicio de departamentos debe tener su propia configuración.

3. Servicio de Empleados (evolución del Reto 1)

Evolucionar el servicio desarrollado en el Reto 1 para que:

- Persista los datos en la base de datos (en lugar de memoria).
- Consulte información del **Servicio de Departamentos** al registrar un empleado.
- Obtenga la configuración de conexión a la base de datos desde variables de entorno.

Endpoints requeridos

Método	Ruta	Descripción
`POST`	`/empleados`	Registra un empleado asociado a un departamento
`GET`	`/empleados/{id}`	Consulta un empleado por su identificador
`GET`	`/empleados`	Lista todos los empleados registrados

Estructura del empleado

```
{
  "id": "string",
  "nombre": "string",
  "email": "string",
  "departamentoId": "string",
  "fechaIngreso": "date"
}
```

Validación requerida

Al registrar un empleado, el servicio debe:

1. Consultar al **Servicio de Departamentos** para verificar que el `departamentoId` existe.
2. Si el departamento no existe, responder con `400 Bad Request` y un mensaje descriptivo.
3. Si el departamento existe, registrar el empleado y responder con `201 Created`.

> La consulta y validación del departamento se debe hacer mediante una petición HTTP REST al servicio de departamentos. Lo anterior con el fin de simular la comunicación básica entre microservicios. En futuros retos se implementarán mecanismos más robustos de comunicación.

Base de datos

- Debe crear una base de datos para el servicio de empleados.
- Debe crear un volumen para la base de datos.
- Debe crear una variable de entorno para la configuración de la base de datos.

Consideraciones importantes

- Utilizar el **nombre del servicio** definido en compose como hostname (ej: `http://departamentos-service:8080/departamento/{id}`).

4. Prepare su servicio para el desastre

Con el fin de garantizar que su servicio sea resiliente, implemente las siguientes mejoras:

- Manejar adecuadamente los errores de comunicación (timeouts, servicio no disponible).

- Implementar reintentos básicos si el servicio de departamentos no está listo.
- Investigar y aplicar patrones de resiliencia como circuit breaker, bulkhead, etc.

5. Documentación con OpenAPI

Para facilitar el consumo y prueba de los servicios, debe incorporar la especificación **OpenAPI (Swagger)** en cada uno de sus microservicios.

Requisitos

- Configurar la librería adecuada según su lenguaje de programación (ej: `Springdoc` para Java/Spring, `Swagger UI Express` para Node.js, `FastAPI` incluye soporte nativo).
- Exponer la interfaz gráfica de Swagger UI (ej: `http://localhost:8080/swagger-ui.html` o `http://localhost:8080/docs`).
- Documentar cada endpoint con:
  - Descripción clara de su propósito.
  - Códigos de respuesta HTTP posibles (200, 201, 400, 404, 500).
  - Estructura de los objetos de entrada y salida (Schemas).

6. Pruebas del Sistema

Verificar el funcionamiento completo del sistema:

Flujo de prueba sugerido

1. Iniciar todos los servicios: `docker-compose up --build`
2. Crear un departamento:

```
curl -X POST http://localhost:8081/departamentos \
  -H "Content-Type: application/json" \
  -d '{"id": "IT", "nombre": "Tecnología", "descripcion": "Departamento de TI"}'
```

3. Crear un empleado asociado al departamento:

```
curl -X POST http://localhost:8080/empleados \
  -H "Content-Type: application/json" \
  -d '{"id": "E001", "nombre": "Juan Pérez", "email": "juan@empresa.com", "departam
```

4. Verificar que el empleado existe: `GET http://localhost:8080/empleados/E001`
5. Intentar crear un empleado con departamento inexistente (debe fallar con 400).
6. Reiniciar los contenedores y verificar que los datos persisten.

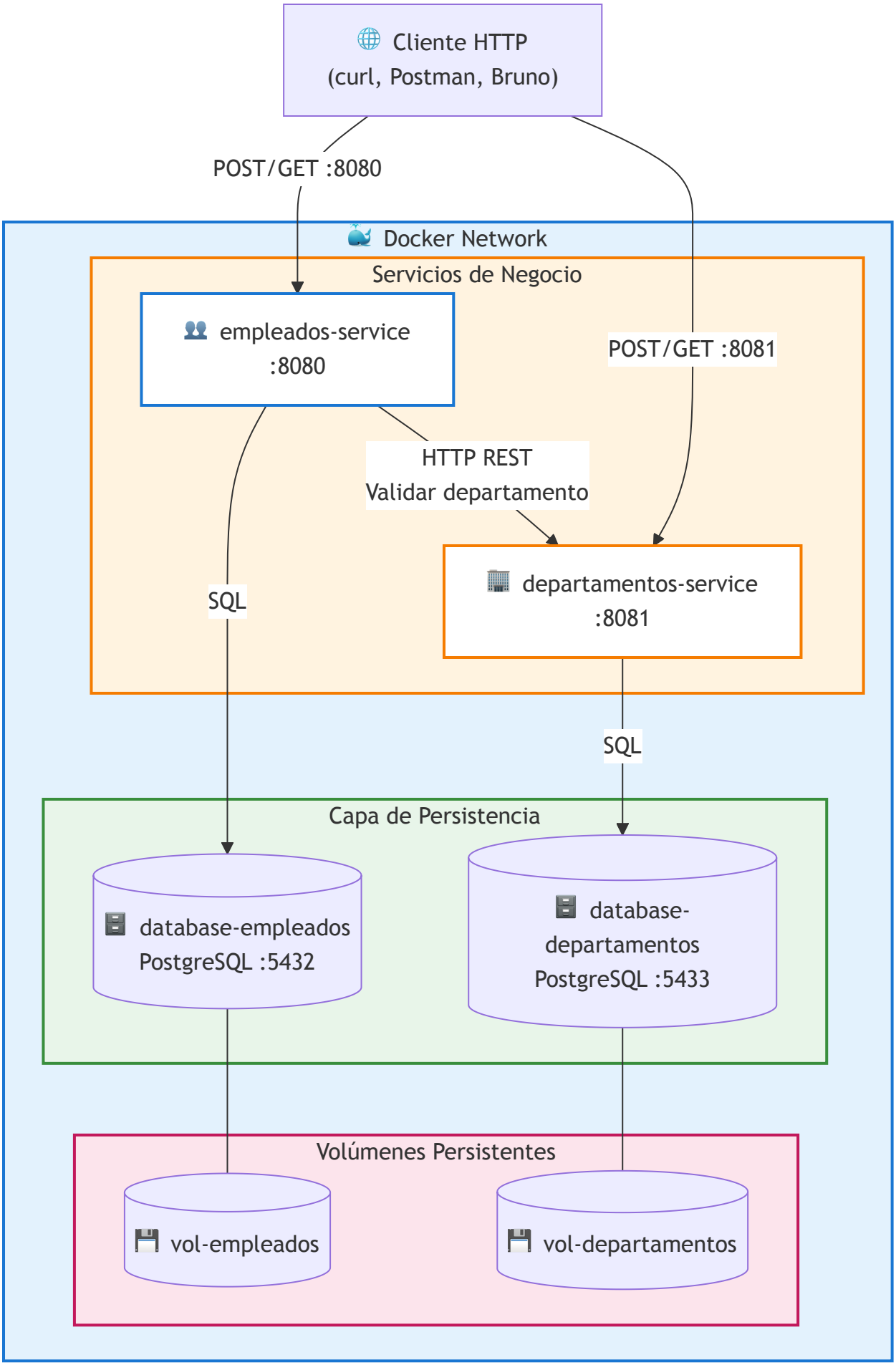
Entregables

- Versione adecuadamente su código en repositorios de GitHub.
- Cree un README.md con instrucciones de despliegue y prueba para cada uno de los servicios.
- Cree un README.md con instrucciones de despliegue para el sistema completo.

Consideraciones

- Cada servicio debe tener su propio `Dockerfile`.
- Cada servicio debe tener su propia base de datos.

Diagrama de Arquitectura Esperada


Cliente HTTP
(curl, Postman, Bruno)
POST/GET :8080
POST/GET :8081
Docker Network
Servicios de Negocio
empleados-service
:8080
HTTP REST
Validar departamento
departamentos-service
:8081
SQL
SQL
Capa de Persistencia
database-empleados
PostgreSQL :5432
database-
departamentos
PostgreSQL :5433
Volúmenes Persistentes
vol-empleados
vol-departamentos

Criterios de Evaluación

El reto se evalúa sobre **5 puntos**, correspondiendo **1 punto** a cada uno de los elementos principales:

#	Elemento	Valor	Aspectos a evaluar
1	**Docker Compose**	1.0	Configuración correcta del `docker-compose.yml` : redes, volúmenes, variables de entorno, dependencias y puertos
2	**Servicio de Departamentos**	1.0	Implementación completa de endpoints, Dockerfile, base de datos, persistencia y **documentación OpenAPI**
3	**Servicio de Empleados**	1.0	Evolución con persistencia, comunicación, validación y **documentación OpenAPI**
4	**Resiliencia**	1.0	Manejo de errores, timeouts, reintentos y aplicación de patrones de resiliencia
5	**Pruebas del Sistema**	1.0	Evidencia de pruebas completas, verificación de persistencia y correcto funcionamiento de **Swagger UI**

> **Nota:** La documentación (README.md) y el versionamiento adecuado en GitHub son requisitos transversales que afectan la evaluación de cada elemento.